

Working with Claude Code in This Project

An onboarding guide for human researchers

Esteban Degetau
World Bank
estebandegetau@gmail.com

Agustín Samano
World Bank
asamanopenaloza@worldbank.org

June 28, 2026

Why this document exists

This project uses Claude Code extensively: for writing pipeline functions, managing a reproducible data workflow, developing measurement instruments, and maintaining documentation. Claude Code can perform most of the technical tasks involved in empirical research. The harder question is what it *should* do versus what the human researcher must own.

This document answers that question. It starts with principles, then maps those principles onto the concrete systems built into this template.

Part 1: Principles

The chain of human ownership

Four responsibilities belong to the human researcher. They form a chain: each depends on the ones before it.

1. Meaning. The human both *receives* and *confers* meaning. Receiving: understanding why the research question matters and what is at stake if the answer is wrong. Conferring: deciding that the model's output — these labels, these estimates, these classifications — *are* what the instrument says they are, and accepting accountability for that claim. An LLM can pattern-match against instrument definitions, but it does not know what the classification means for causal identification or external validity. It does not understand that getting this wrong could invalidate the downstream analysis. Knowledge does not exist until someone claims it and can be held accountable.

2. Learning. The human must learn enough to own the work. This is not a moral imperative but a practical one. If you defer all intellectual engagement to the end, you face a misaligned incentive to share output you do not understand. The project infrastructure should be a commitment device that forces intellectual engagement at the right moments — not everywhere, but at every point central to the research contribution.

A useful heuristic: **how central is this task to the contribution?** Data pipeline plumbing is instrumental; delegate it fully and verify outputs against your

mental model. But instrument design is the intellectual contribution. You must understand in detail how and why the instruments work, and when they might not. Delegate the instrumental. Own the core.

3. Credibility. The depth of your involvement determines the depth at which you can credibly defend your work. You need to have lived through the iteration history — the failures, the diagnostic decisions, the tradeoffs chosen and why — to defend them under peer review. The iteration log makes that history legible. You choose where to go deep, but you own the consequences.

4. Decision ownership. Claude can analyze, propose, challenge, and trace implications. But the commit (“we will do this, not that”) belongs to the human. Decisions are auditable in ways that judgment is not. The iteration log records every decision with a date, context, and rationale. The person who made the decision faces the consequences if it was wrong.

The core heuristic

AI owns tasks where errors are recoverable. Humans own tasks where errors compound silently.

Code has tests. Metrics can be recomputed. YAML can be re-validated. If Claude gets these wrong, you find out quickly and fix it cheaply.

A bad research design does not throw an error. A wrong interpretation gets baked into the next iteration’s decision field and shapes all future work. A cost incurrence cannot be un-spent. The infrastructure in this template forces human involvement precisely at these high-consequence points.

What honest research with AI looks like

The mechanisms built into this template are tools for honesty:

- The **iteration log** records failures, not just successes. It is how reviewers know you actually tested the instrument rather than picking the first version that worked.
- The **delta log** forces you to acknowledge when implementation contradicts your plan, rather than quietly drifting.
- **Strategy sync** makes you justify decisions under adversarial questioning, not rationalize them after the fact.
- **One change at a time** prevents hiding behind a bundle of changes where you cannot tell what worked.
- The **pre-flight checklist** prevents running experiments on uncommitted code where you cannot reproduce the result.

These are not efficiency tools. They are integrity infrastructure.

Part 2: Architecture

The strategy document as constitution

`docs/strategy.md` is the single source of truth for research methodology. Every agent, skill, and review process references it. It specifies your measurement instruments, validation pipeline, success criteria, and implementation sequencing.

The strategy document lives in a **three-tier documentation system** that enforces the separation between human specification and machine context:

- **Tier 1 (auto-update):** `CLAUDE.md` files. These are execution context for Claude Code. Claude updates them directly when implementation changes make them stale. They track current status, file inventories, and pipeline definitions.
- **Tier 2 (never auto-edit):** `docs/strategy.md` and any other documents designated Tier 2 in your `CLAUDE.md`. These represent deliberate research design decisions. Claude never edits them directly. Instead, bottom-up discoveries are logged as entries in `docs/deltas.md`, where they accumulate until you reconcile them via `/strategy-sync`.
- **Tier 3 (ignore):** Stable reference documents, generated artifacts, and infrastructure docs that rarely need updates.

Why this matters: You own the research design. The AI owns the execution context. Changes flow upward through the delta log, never through autonomous edits to the strategy.

The agent system

The project uses specialized Claude Code agents in `.claude/agents/`, organized by function and deliberately constrained:

- **Read-only agents** (limited to `Read`, `Grep`, `Glob`): reviewers, domain specialists, verifiers. A reviewer that can also edit files is a developer pretending to review.
- **Write-capable agents:** developers, builders, documenters.
- **Model tiering:** cheap fast models for narrow tasks; more capable models for tasks requiring reasoning across domains.

See `COMPANION.md` for the full agent taxonomy and design principles. See `CLAUDE.md` for your project-specific agent configurations.

The skill system

Skills are procedures codified as prompts, invoked via slash commands. They enforce a repeatable workflow and prevent steps from being skipped.

Skill	When to use	What it enforces
<code>/setup-project</code>	Once, from template	Onboarding interview → draft <code>CLAUDE.md</code>
<code>/doc-sync</code>	After significant work	Tier 1 updates + delta logging + nudge at 3+ deltas

Skill	When to use	What it enforces
<code>/strategy-sync</code>	Stage gates or 3+ deltas	Adversarial reconciliation
<code>/log-iteration</code>	After pipeline stage	Auto-gathered metadata + human interpretation/decision
<code>/progress-report</code>	External communication	Auto-gathered state + terminology translation
<code>/quarto-style</code>	Writing <code>.qmd</code> files	Formatting authority

The pipeline

Two pipeline rules are absolute:

1. **All data generation goes through the pipeline.** No standalone scripts, no manual `saveRDS()` or `write_csv()` outside the pipeline tool. Reproducibility requires tracked dependencies.
2. **No autonomous API calls.** Claude never runs the pipeline on API-calling targets without explicit human approval. Read-only operations are always safe.

Part 3: The Iteration Cycle

The daily workflow

The core loop for instrument development:

1. Edit instrument definition (one component at a time)
2. `/pre-flight` → verify everything is ready
3. `tar_make(<target>)` → human authorizes the run
4. `/review-iteration` → structured diagnosis (read-only)
5. Discuss results → human interprets, decides next step
6. `/log-iteration` → record what happened and why
7. Repeat or advance to next stage

Steps 1, 3, 5, and 7 are human decisions. Steps 2, 4, and 6 are Claude-assisted procedures that enforce rigor without taking control.

The iteration log as lab notebook

If you have worked in Stata or R, you know the problem: you changed a parameter three weeks ago, results improved, and you cannot reconstruct what you changed or why. The iteration log (`prompts/iterations/<instrument>.yaml`) is that lab notebook. Every pipeline run records: instrument version, model, metrics, your interpretation, your decision, and a git hash linking to the exact file state. Reconstruct any past result with `git show <hash>:<instrument_file>`.

The documentation lifecycle

Implementation discoveries feed back into the research design through a structured cycle:

```
docs/strategy.md (human intent)
  ↓ implementation work
docs/deltas.md (bottom-up discoveries via /doc-sync)
  ↓ accumulation trigger (3+ entries or stage gate)
/strategy-sync (adversarial reconciliation)
  ↓ disposition: incorporate / acknowledge / defer
docs/strategy.md (updated) + audit trail (rationale recorded)
```

The strategy document stays alive rather than becoming a fossil from week one.

Exploration vs. production runs

Not all pipeline runs are equal:

- **Exploration runs** use cheap models to test infrastructure, debug mechanics, and estimate costs. Results are not formally reportable.
- **Production runs** use the validated model defined in your `CLAUDE.md` for formal evaluation. Results feed the iteration log.

The iteration log marks each run's model and provider. Mixing providers across iterations invalidates stage comparability.

How to populate docs/strategy.md

Describe your project to Claude in plain English. Ask Claude to draft a structured plan following the section headers in `docs/strategy.md`. Review it carefully — the draft will be wrong in important ways, and correcting it is the most valuable intellectual work you can do at the start of the project. Your corrections encode your understanding of the problem, your methodological choices, and the constraints that make those choices non-obvious.

The 10 workflow conventions

These rules prevent recurring friction patterns. Each exists because someone learned the hard way.

1. **Plan-first mode.** When asked to diagnose or investigate, Claude presents findings and waits. No implementation unless explicitly told to. *Why: prevents fixing things before you understand the problem.*
2. **Root cause first.** Identify the cause before proposing a fix. Do not patch symptoms. *Why: a bad instrument example causes a test failure. Improving the test hides the problem.*
3. **Model ID validation.** Verify model IDs against the list in `CLAUDE.md` before writing them. *Why: stale IDs cause silent pipeline failures.*
4. **Prefer existing files.** Search before creating. *Why: duplicate implementations create maintenance burden.*

5. **No autonomous API calls.** Claude never runs API-calling targets without human approval. *Why: API calls cost money and change pipeline state.*
6. **Commit before pipeline runs.** No uncommitted changes to instrument files or pipeline functions. *Why: the iteration log stores git hashes. Uncommitted changes break reproducibility.*
7. **One change at a time.** One instrument component per iteration. *Why: if three things change and a metric improves, you cannot tell which change helped.*
8. **Pipeline data validation.** Verify result shape after a run before proceeding. *Why: catches silent data issues before they propagate.*
9. **Quarto render safety.** Render specific files, never the full project. *Why: prevents accidentally re-rendering expensive notebooks.*
10. **Strategy reconciliation.** Run `/strategy-sync` at stage gates or when 3+ unresolved deltas accumulate. *Why: implementation drift is inevitable. Periodic reconciliation forces you to justify decisions on the record.*

Part 4: Reference

Key file locations

Path	Purpose
<code>COMPANION.md</code>	Permanent principles and architecture (Tier 2)
<code>CLAUDE.md</code>	Project-specific instructions for Claude Code (Tier 1)
<code>docs/strategy.md</code>	Authoritative research methodology (Tier 2)
<code>docs/deltas.md</code>	Bottom-up implementation discoveries
<code>prompts/iterations/<instrument>.yaml</code>	Iteration log per instrument
<code>.claude/agents/</code>	Agent configurations
<code>.claude/skills/</code>	Skill definitions
<code>reports/</code>	Dated progress reports

Skill quick reference

Command	When to use
<code>/setup-project</code>	Once, when starting from template
<code>/pre-flight</code>	Before any API-calling pipeline run

Command	When to use
<code>/review-iteration</code>	After a pipeline stage, before deciding what to change
<code>/log-iteration</code>	After discussing results, to record the iteration
<code>/doc-sync</code>	After significant implementation work
<code>/strategy-sync</code>	At stage gates or when 3+ deltas accumulate
<code>/progress-report</code>	When preparing updates for external stakeholders

The iteration log schema

```
- iteration: 1
  instrument: "<instrument_id>"
  instrument_version: "0.1.0"
  date: "YYYY-MM-DD"
  git_commit: "0ecc0db"
  model: "<model_id>"
  provider: "anthropic"
  stage: "s1"
  changes: >
    What changed in the instrument definition since last iteration.
  results:
    overall_pass: false
    metrics:
      - test: "I_legal_outputs"    # S1 format uses test:
        value: 1.0000
        threshold: 1.0000
        pass: true
      - metric: "primary_metric"  # S2/S3 format uses metric:
        value: 0.0000
        target: 0.0000
        pass: false
    interpretation: >
      What these results mean. Written by the human.
  decision: >
    What to do next. Decided by the human.
```

Every past instrument version is recoverable: `git show <hash>:prompts/<instrument_file>`.